

Northeastern University DS 5500 Capstone

AI Teaching Assistant for Faculty in a Real Course

Methodology Report

Anjana Deivasigamani

Priyadharshan Sengutuvan

Raghu Ram Baskaran

Khoury College of Computer Sciences, Northeastern University

March 2026

Contents

1	Introduction	3
2	Purpose of the Methodology	4
3	Problem Statement	5
3.1	Problem Definition	5
3.2	Significance and Related Work	6
3.3	Related Work	7
4	Data Collection and Preparation	8
4.1	Data Sources	8
4.2	Dataset Description	9
4.3	Preprocessing Steps	9
5	Selection of Machine Learning and LLM Models	11
5.1	Models Considered	11
5.2	Final Model Selection	12
6	Model Development and Training	12
6.1	At-Risk Classification: Architecture and Training	12
6.2	RAG Pipeline: Architecture and Configuration	13

1 Introduction

This report presents the methodology underlying the development of an AI Teaching Assistant designed to augment faculty support within Canvas Learning Management System (LMS) environments. The system combines Retrieval-Augmented Generation (RAG), learning analytics, and automated at-risk student detection to transform raw LMS interaction data into actionable instructional insights.

Our approach integrates four complementary methodological pillars: (1) a **RAG pipeline** that grounds AI-generated responses in verified course materials; (2) a **learning analytics engine** that applies rule-based and machine learning models to student performance data; (3) a **Quiz Builder module** that leverages the RAG knowledge base to automatically generate assessment questions targeted at identified misconception areas; and (4) a **full-stack production architecture** built on Google Cloud Platform (GCP) that delivers these capabilities through an interactive faculty dashboard. Together, these components address the gap between data-rich LMS environments and the actionable intelligence faculty need to intervene early, generate assessments efficiently, and personalize instruction at scale.



Figure 1: *End-to-end system architecture of the AI Teaching Assistant. Data flows from Canvas LMS and course PDFs through an ingestion pipeline into dual storage (PostgreSQL for structured data, ChromaDB for vector embeddings), powering a RAG engine and at-risk detection module that serve a React faculty dashboard via FastAPI on GCP.*

2 Purpose of the Methodology

Faculty in large enrollment courses face a fundamental scalability problem: Canvas and similar LMS platforms collect granular interaction data—submission timestamps, quiz attempts, discussion frequency—yet provide no intelligent layer for synthesizing these signals into early-warning

alerts or content-grounded answers. This methodology is designed to close that gap through two coordinated approaches.

First, we employ a **RAG-based question-answering system** rather than a purely parametric language model. This choice is motivated by the need for factual grounding: a GPT-4 model without retrieval may hallucinate course-specific details, whereas a RAG system retrieves verified excerpts from the professor’s own slides and notes before generating a response. Comparing naive LLM responses against RAG-augmented responses allows us to quantify this grounding benefit using ROUGE-L and answer faithfulness scores.

Second, we compare a **rule-based risk detector** (threshold logic on grades and submission patterns) against a **supervised ML classifier** (Logistic Regression and Random Forest) to determine how much predictive lift a learned model provides over expert heuristics. This comparison is methodologically necessary because it informs whether training overhead is justified given the modest dataset size available at a single institution.

Third, the **Quiz Builder** extends the RAG pipeline into the assessment generation domain. Rather than treating question generation as an independent task, the module retrieves course passages most associated with flagged misconception areas and conditions GPT-4o to produce well-formed questions with answer keys and difficulty tags. This approach ensures generated assessments remain grounded in actual course content and are directly responsive to observed student performance gaps, closing the feedback loop between analytics and instruction.

3 Problem Statement

3.1 Problem Definition

This project addresses two interrelated machine learning sub-problems within the domain of educational technology:

- (1) **At-risk Student Classification** — Given a student’s rolling LMS interaction features (assignment submission rate, grade trajectory, discussion activity), predict a binary risk label (at-risk / not at-risk) with sufficient recall to support timely intervention. This is a *supervised binary classification* task.
- (2) **Course-Grounded Question Answering** — Given a natural language query from a faculty member or student, retrieve the most relevant passage from the course knowledge base and generate a coherent, faithful answer. This is a *retrieval-augmented generation* task combining dense passage retrieval with large language model (LLM) inference.
- (3) **Automated Quiz Generation** — Given a course topic or a set of identified student misconceptions, automatically generate a contextually relevant set of assessment questions from the RAG knowledge base. This is a *conditional text generation* task where question quality is optimized for curriculum coverage and pedagogical alignment with instructor-defined learning objectives.

3.2 Significance and Related Work

The significance of this work is grounded in three dimensions. Academically, it contributes to the growing literature on *learning analytics* and *intelligent tutoring systems (ITS)*. Research has shown that early identification of at-risk students combined with personalized feedback can measurably improve retention and academic performance [1]. Industry surveys indicate that over 80% of faculty at research-intensive universities report lacking adequate tools to monitor individual student progress in real time [2].

From an AI systems perspective, this project addresses a known limitation of general-purpose LLMs: their inability to reliably cite course-specific, proprietary content. The RAG paradigm [3] has emerged as the canonical solution, and validating its effectiveness in an educational context contributes empirical evidence to this body of work.

Finally, the real-world impact is direct: a deployed system at Northeastern University could benefit hundreds of students per semester by enabling faculty to shift from reactive grading to proactive intervention.

3.3 Related Work

AI Teaching Assistant Adoption in University Education (2025). This paper presents a faculty-facing AI teaching assistant that assists instructors by drafting responses to student questions and managing repetitive queries under a human-in-the-loop workflow. The study demonstrates that AI assistants are feasible and valuable when instructor transparency and oversight are maintained. However, the system lacks a RAG pipeline over course materials, provides no learning analytics dashboard, and does not support adaptive assessment features such as quiz generation. These limitations directly motivate the design of our system, which integrates all three capabilities.

PQSCT: Pseudo-Siamese BERT for Concept Tagging (IEEE TLT, 2023). This work proposes a pseudo-siamese BERT architecture that jointly encodes question text and solution text for multi-label concept classification. Results show that modeling both question and solution context substantially improves concept tagging accuracy over question-only baselines. While the approach demonstrates transformer-based support for concept-level analytics, it does not extend to system-level integration, instructor dashboards, or feedback generation. Our system builds on this insight by incorporating misconception tagging into the instructor dashboard to surface common student knowledge gaps.

Automatic Question Group Generation for Practical Educational Applications (IEEE TLT, 2024). This paper proposes a two-stage hybrid approach for generating complete assessment sets: a transformer-based model generates candidate questions from instructional content, and a genetic algorithm then optimizes the group for coverage, redundancy, and difficulty balance. The generated question groups outperform baseline methods in instructional quality. This work directly informs our planned **Quiz Builder** feature, which will allow faculty to automati-

cally generate contextually relevant quiz questions from uploaded course materials using the RAG knowledge base. Unlike the above work, our system will integrate student performance signals to bias question generation toward topics where misconceptions are most prevalent.

Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks (2020). Lewis et al. [3] introduce the RAG framework, which combines a dense document retriever with a sequence-to-sequence generator, significantly reducing hallucinations by grounding responses in retrieved passages. This foundational architecture forms the core of our question-answering and quiz generation pipeline. While the original work is not education-specific, our adaptation aligns retrieval with course learning objectives and instructor-defined rubrics, addressing the pedagogical gaps noted in the original paper.

4 Data Collection and Preparation

4.1 Data Sources

The project draws on two distinct data sources:

Course Knowledge Base. We utilize 62 PDF lecture slides and notes from CMU’s 15-445/645 Database Systems course (Spring 2025), publicly released by Professor Andy Pavlo. This corpus was selected because it is (a) rigorously curated over ten years of instruction, (b) domain-specific with consistent terminology, and (c) representative of a typical university computer science course. The materials were downloaded programmatically via the course website and stored in a Google Cloud Storage bucket.

Student Interaction Data. In the absence of real identifiable student records (due to FERPA constraints), we construct a synthetic interaction dataset inspired by the Open University Learning Analytics Dataset (OULAD) [4] and the EdNet dataset [5]. Synthetic student records are generated using realistic statistical distributions (submission delay modeled as a log-normal distribution;

grade trajectories modeled with Markov chains calibrated on published at-risk prevalence rates of approximately 15–20%).

4.2 Dataset Description

Course Knowledge Base:

- 62 PDF files, approximately 1,297 pages total.
- After chunking, the corpus yields 1,000–1,200 text segments.
- Chunks cover topics including: relational algebra (18%), SQL query processing (22%), indexing and storage (20%), transaction management (15%), concurrency control (12%), and distributed databases (13%).
- Supplemented with 80 manually curated Q&A pairs and 40–50 annotated error patterns for evaluation.

Synthetic Student Interaction Dataset:

- 500 simulated student records across 15 assignments and 5 quizzes.
- Features: `submission_rate`, `mean_grade`, `grade_slope` (linear regression over time), `days_since_active`, `discussion_post_count`, `late_submission_count`.
- Class distribution: approximately 17% at-risk (intentional imbalance reflecting real-world prevalence).
- No personally identifiable information (PII) is used at any stage.

4.3 Preprocessing Steps

PDF Processing Pipeline. Each PDF is processed through the following stages:

1. **Text extraction:** PyMuPDF (`fitz`) extracts raw text from each page, preserving layout order.
2. **Cleaning:** Header/footer artifacts, page numbers, and broken hyphenation are removed using regular expressions.
3. **Chunking:** Text is segmented into overlapping windows of 500 tokens with a 50-token overlap, using a sliding-window strategy implemented in LangChain's `RecursiveCharacterTextSplitter`.
4. **Embedding:** Each chunk is embedded using OpenAI's `text-embedding-3-small` model (1,536-dimensional vectors).
5. **Deduplication:** MD5 hashing of chunk content prevents redundant storage when PDFs are re-uploaded.
6. **Storage:** Embeddings and metadata (source filename, page number) are persisted to ChromaDB.

Student Data Pipeline. The synthetic tabular dataset undergoes:

1. **Missing value imputation:** Median imputation for grade features; zero-fill for count features.
2. **Normalization:** Min-max scaling applied to all numeric features to ensure gradient-based models converge.
3. **Class imbalance handling:** SMOTE (Synthetic Minority Over-sampling Technique) applied to the training set to address the 17% minority class.
4. **Train/validation/test split:** 70% / 15% / 15% stratified split preserving class ratios.

Automation. A background watcher script (`watch_pdfs.py`) monitors the GCS bucket for new uploads and triggers the full PDF ingestion pipeline automatically, enabling continuous knowledge base updates without manual intervention.

5 Selection of Machine Learning and LLM Models

5.1 Models Considered

For At-Risk Detection:

We considered the following model families, evaluated on three criteria: interpretability (required for faculty trust), performance on small datasets, and training speed.

- **Rule-based threshold classifier** — Baseline. Flags a student if submission rate < 0.6 OR mean grade < 65 . Simple and fully interpretable.
- **Logistic Regression** — Linear probabilistic classifier. Interpretable coefficients, fast training, suitable for small datasets.
- **Random Forest** — Ensemble of decision trees. Handles non-linear feature interactions; provides feature importance rankings.
- **Gradient Boosting (XGBoost)** — Higher capacity ensemble. Considered but deprioritized due to reduced interpretability relative to modest performance gains at our dataset scale.
- **Deep learning (LSTM/Transformer on sequences)** — Considered for modeling temporal grade trajectories. Excluded for this iteration due to data volume requirements; planned for future work.

For Question Answering / RAG:

- **Naive GPT-4 (no retrieval)** — Baseline. Answers questions purely from parametric knowledge. Fast, but prone to hallucinating course-specific details.
- **RAG with GPT-4 + ChromaDB** — Primary system. Retrieves top- k relevant chunks via cosine similarity search, prepends them as context, then generates an answer.

- **RAG with open-source LLM (Llama 3)** — Considered for cost efficiency. Preliminary experiments showed lower faithfulness scores; excluded from production system.

5.2 Final Model Selection

For risk detection, **Logistic Regression with SMOTE** is selected as the primary model based on its superior recall on the minority class (target: $\text{recall} \geq 0.80$), interpretable coefficients, and robust performance relative to dataset size. Random Forest is retained as a comparison model to assess non-linear feature interaction benefits.

For question answering, the **RAG pipeline with GPT-4o and ChromaDB** is selected. The key justification is the requirement for *answer faithfulness* — responses must be traceable to specific course materials. Evaluation against the 80 manually curated Q&A pairs confirms a ROUGE-L improvement of approximately 18 points over naive GPT-4 generation.

Evaluation metrics used for model selection:

Table 1: Evaluation Metrics by Sub-problem

Sub-problem	Primary Metric	Secondary Metrics
At-risk classification	Recall (minority class)	F1-score, Precision, AUC-ROC
Question answering	ROUGE-L	Answer Faithfulness, Latency

6 Model Development and Training

6.1 At-Risk Classification: Architecture and Training

The classification model receives a 6-dimensional feature vector per student: $\mathbf{x} = [\text{submission_rate}, \text{mean_grade}, \text{gpa}, \text{credits_completed}, \text{credits_attempted}, \text{credits_dropped}]$.

Logistic Regression is configured with L2 regularization ($\lambda = 0.1$) to prevent overfitting on

the training set. The decision threshold is tuned on the validation set to maximize recall while maintaining precision above 0.60, reflecting the asymmetric cost of missing an at-risk student.

Random Forest uses 100 estimators with maximum depth of 10, trained with `class_weight="balanced"` as an alternative to SMOTE. Feature importance scores are extracted to provide interpretable explanations for individual risk flags.

Dataset split: 70% training (350 records), 15% validation (75), 15% test (75), stratified by class label. Overfitting is monitored via learning curves; cross-validation (5-fold) is applied on the training set.

6.2 RAG Pipeline: Architecture and Configuration

The RAG pipeline follows a standard *retrieve-then-generate* architecture:

1. **Query embedding:** The incoming natural language query is embedded using `text-embedding-3-small` producing a 1,536-dimensional query vector.
2. **Retrieval:** ChromaDB performs approximate nearest-neighbor search using cosine similarity, returning the top- $k = 5$ most relevant chunks.
3. **Context assembly:** Retrieved chunks are concatenated with the query into a structured prompt template that instructs the model to answer only from the provided context and acknowledge uncertainty when the answer cannot be found.
4. **Generation:** GPT-4o generates the final response with a maximum of 512 output tokens.

No fine-tuning is performed on the language model; the system relies entirely on in-context learning, which is appropriate given the domain coverage achievable through retrieval alone.

References

- [1] R. S. Baker and P. S. Inventado, “Educational data mining and learning analytics,” *Learning Analytics: From Research to Practice*, pp. 61–75, 2014.
- [2] EDUCAUSE, “EDUCAUSE horizon report: Teaching and learning edition,” EDUCAUSE, 2023. [Online]. Available: <https://library.educause.edu/resources/2023/6/2023-educause-horizon-report>.
- [3] P. Lewis, E. Perez, A. Piktus, et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [4] J. Kuzilek, M. Hlosta, and Z. Zdrahal, *Open university learning analytics dataset*, Scientific Data, 2017. DOI: 10.1038/sdata.2017.171.
- [5] Y. Choi, Y. Lee, J. Cho, et al., “EdNet: A large-scale hierarchical dataset in education,” *Proceedings of the International Conference on Artificial Intelligence in Education*, pp. 69–73, 2020.